ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS

ASIGNATURA:	BASE DE DATOS
PROFESOR:	ING YADIRA FRANCO
PERÍODO ACADÉMICO:	2026-A

PROYECTO

TÍTULO

Sistema Centralizado de Gestión Comercial, Inventario y Logística para LEJOS-TECH

ESTUDIANTES:

MATEO JOSUE RODRÍGUEZ QUEVEDO

LEONEL ALEJANDRO MOREIRA RENGIFO

ÍNDICE

1. Introducción---------- 3
2. Propósito de la práctica---------- 3
3. Objetivo General---------- 3
4. Objetivos Específicos---------- 3
5. Instrucciones---------- 4
    5.1 Implementación obligatoria---------- 4
6. Desarrollo---------- 5
    6.1 Análisis del negocio---------- 5
        6.1.1 Descripción del problema.---------- 5
        6.1.2 Actores---------- 5
        6.1.3 Reglas del negocio---------- 5
        6.1.3 Diccionario de datos---------- 7
        6.1.4 Validaciones---------- 7
    6.2 Diseño de análisis---------- 8
        6.2.1 Modelo Entidad Relación (Mer)---------- 8
        6.2.2 Normalización---------- 8
        6.2.3 Modelo Lógico---------- 9
        6.2.4 Modelo Físico---------- 10
        6.2.5 Consultas---------- 10
        6.2.6 Usuarios y roles---------- 11
        6.2.7 Vistas---------- 11
        6.2.8 Funciones---------- 12
        6.2.9 Procedimientos almacenados---------- 12
        6.2.10 Triggers---------- 13
        6.2.11 Transacciones---------- 13
        6.2.12 Seguridad y respaldos---------- 14
7. Presentación---------- 14
    7.1 Informe PDF---------- 14
    7.2 Repositorio GitHub---------- 14
    7.3 Base de datos en la nube---------- 14
    7.4 Defensa oral---------- 15
8. Recursos necesarios---------- 15
9. Conclusiones---------- 16
10. Recomendaciones---------- 16
11. Bibliografía---------- 18
12. Anexos---------- 18

1. Introducción

El presente proyecto tiene como finalidad diseñar e implementar una base de datos relacional para LEJOS-TECH, empresa dedicada a la venta de componentes para computadoras. La propuesta nace de la necesidad de contar con un sistema que permita gestionar de manera eficiente el inventario en un entorno caracterizado por la constante actualización tecnológica y la variación de precios de los productos. Mediante una estructura organizada y confiable para el almacenamiento de la información, el proyecto busca optimizar de la mejor manera el control de existencias, mejorar la administración de clientes y proveedores, agilizar los procesos internos y proporcionar información precisa que facilite la toma de decisiones dentro de la organización.

2. Propósito de la práctica

Diseñar e implementar una base de datos relacional para la empresa LEJOS-TECH que permite organizar y administrar de manera eficiente la información relacionada con el inventario de los componentes de hardware y las ventas de equipos ensamblados. Con ello se busca solucionar los problemas de control existentes, mejorar los precios de las ventas e instalaciones y garantizar la integridad y confiabilidad de los datos para facilitar la gestión dentro de la empresa.

3. Objetivo General

Desarrollar y administrar un sistema para "LEJOS-TECH" mediante la implementación de una base de datos relacional para optimizar los procesos de facturación, control de inventario para todos los servicios y componentes tecnológicos.

4. Objetivos Específicos

- Identificar las entidades, atributos y relaciones del modelo de negocio de LEJOS-TECH.
- Elaborar el Modelo Entidad-Relación (MER).
- Aplicar las reglas de normalización hasta la Tercera Forma Normal (3FN).
- Implementar el modelo físico mediante scripts DDL en MySQL.
- Desarrollar consultas avanzadas, vistas, funciones, procedimientos almacenados y triggers adaptados a la venta de hardware.

- Configurar la seguridad mediante roles de usuarios y establecer políticas de respaldo.

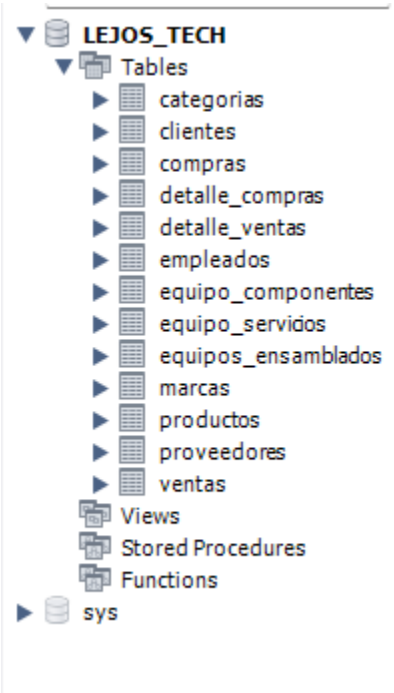
5. Instrucciones

Para el proyecto se rige a una estructura especial la cual se basa en:

Fase	Entregable
Análisis del Negocio	Descripción del problema, actores, reglas de negocio, diccionario de datos
Diseño Conceptual	MER con entidades, atributos, cardinalidades y relaciones
Normalización	Tabla sin normalizar → 1FN → 2FN → 3FN con explicación de cambios
Diseño Lógico	Tablas, campos, tipos de datos, PKs, FKs y relaciones
Diseño Físico	Scripts de creación de BD, tablas, restricciones y reglas CASCADE/RESTRICT

5.1 Implementación obligatoria

- Número mínimo de tablas = 10.



- Registros mínimos por tabla mínima 50.

```sql
-- Datos: equipo_servicios (113 registros)
INSERT INTO equipo_servicios (servicio_id, equipo_id, empleado_id, fecha_servicio, costo, descripcion)
  (1, 1, 17, '2025-08-28', 18.24, 'Ensamblaje e instalacion de sistema operativo'),
  (2, 1, 41, '2023-08-07', 45.03, 'Optimizacion y limpieza de software'),
  (3, 2, 22, '2024-06-21', 14.06, 'Optimizacion y limpieza de software'),
  (4, 2, 18, '2024-03-19', 63.51, 'Optimizacion y limpieza de software'),
  (5, 3, 1, '2024-04-23', 71.39, 'Mantenimiento preventivo'),
  (6, 3, 9, '2023-07-17', 51.21, 'Instalacion de componentes adicionales'),
  (7, 3, 5, '2023-05-10', 41.91, 'Configuracion de red y perifericos'),
  (8, 4, 46, '2023-07-21', 27.91, 'Configuracion de red y perifericos'),
  (9, 4, 40, '2023-10-02', 56.34, 'Ensamblaje e instalacion de sistema operativo'),
  (10, 5, 33, '2023-05-04', 64.93, 'Optimizacion y limpieza de software'),
  (11, 6, 16, '2023-01-22', 16.23, 'Ensamblaje e instalacion de sistema operativo'),
  (12, 6, 37, '2026-03-03', 64.57, 'Ensamblaje e instalacion de sistema operativo'),
  (13, 7, 8, '2023-01-01', 69.64, 'Instalacion de componentes adicionales'),
  (14, 7, 22, '2025-08-20', 28.78, 'Ensamblaje e instalacion de sistema operativo'),
  (15, 7, 24, '2025-03-04', 38.05, 'Optimizacion y limpieza de software'),
  (16, 8, 2, '2024-08-16', 17.91, 'Optimizacion y limpieza de software'),
  (17, 8, 29, '2023-02-09', 55.85, 'Ensamblaje e instalacion de sistema operativo'),
  (18, 8, 46, '2024-11-28', 33.68, 'Configuracion de red y perifericos'),
  (19, 9, 22, '2025-01-17', 25.59, 'Optimizacion y limpieza de software'),
  (20, 10, 39, '2025-09-18', 24.48, 'Optimizacion y limpieza de software'),
  (21, 11, 7, '2023-03-16', 19.83, 'Optimizacion y limpieza de software'),
  (22, 11, 7, '2025-10-07', 23.13, 'Optimizacion y limpieza de software'),
  (23, 11, 14, '2026-04-23', 33.66, 'Optimizacion y limpieza de software'),
  (24, 12, 48, '2023-09-14', 68.72, 'Optimizacion y limpieza de software'),
  (25, 12, 33, '2024-12-19', 22.16, 'Instalacion de componentes adicionales'),
  (26, 12, 7, '2026-11-20', 52.43, 'Mantenimiento preventivo'),
```

- Consultas y subconsultas mínimas en MYSQL = 20

```sql
-- Consulta 1: Catalogo completo de productos con categoria, marca y proveedor
SELECT p.producto_id, p.nombre, c.nombre AS categoria, m.nombre AS marca,
       pr.nombre AS proveedor, p.precio_venta, p.stock
FROM productos p
JOIN categorias c  ON p.categoria_id = c.categoria_id
JOIN marcas m       ON p.marca_id = m.marca_id
JOIN proveedores pr ON p.proveedor_id = pr.proveedor_id
ORDER BY p.nombre;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

producto_id	nombre	categoria	marca	proveedor	precio_venta	stock
34	Adaptador USB-C ADATA	Software Gama Media	ADATA	Distribuidora Rodriguez S.A.	796.52	19
70	Adaptador USB-C Apple	Parlantes Gama Media	Apple	Distribuidora Medina Tech Supply	1046.84	23
63	Audifonos Gamer 7.1 Apple	Audifonos Gama Media	Apple	Distribuidora Cruz Global	909.93	19
27	Audifonos Gamer 7.1 MSI	Audifonos Gama Media	MSI	Distribuidora Castro Global	205.80	106
33	Cable HDMI 2.1 EVGA	Parlantes Gama Media	EVGA	Distribuidora Vega Global	721.31	98
69	Cable HDMI 2.1 Razer	Sillas Gamer Gama Media	Razer	Distribuidora Villacis S.A.	214.12	71
57	Cooler Aire HP	Consolas Gama Alta	HP	Distribuidora Bravo S.A.	75.46	89
21	Cooler Aire Lenovo	Memorias RAM Gama Media	Lenovo	Distribuidora Guerrero S.A.	484.97	81
22	Cooler Liquido 240mm D-Link	Procesadores Gama Alta	D-Link	Distribuidora Diaz Cia Ltda	135.63	82
58	Cooler Liquido 240mm Genius	Switches Gama Alta	Genius	Distribuidora Villacis Global	1025.16	31
53	Fuente de Poder 650W Acer	Fuentes de Poder Gama M...	Acer	Distribuidora Reyes S.A.	921.58	108
17	Fuente de Poder 650W Corsair	Tarjetas de Video Gama M...	Corsair	Distribuidora Cabrera Global	621.07	79
18	Fuente de Poder 750W Hyp...	Routers Gama Media	HyperX	Distribuidora Barros Tech Supply	341.84	47
54	Fuente de Poder 750W Net...	Parlantes Gama Alta	Netgear	Distribuidora Yepez Global	182.54	32
19	Gabinete ATX RGB Apple	Audifonos Gama Media	Apple	Distribuidora Guerrero Import E...	340.50	38
55	Gabinete ATX RGB Seagate	Tablets Gama Alta	Seagate	Distribuidora Yepez Import Export	526.09	57

Result 1 | Read Only

```sql
-- Consulta 2: Total comprado y numero de ventas por cliente
SELECT cl.cliente_id, cl.nombre, COUNT(v.venta_id) AS num_ventas, SUM(v.total) AS total_comprado
FROM clientes cl
JOIN ventas v ON cl.cliente_id = v.cliente_id
GROUP BY cl.cliente_id, cl.nombre
ORDER BY total_comprado DESC;

-- Consulta 3: Empleados que han realizado al menos una venta
SELECT DISTINCT e.empleado id, e.nombre, e.cargo
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cliente_id	nombre	num_ventas	total_comprado
32	Karina Yepez	6	15281.20
11	Miguel Castro	5	12067.27
45	Tomas Aguilar	3	10751.30
13	Ricardo Pazmiño	2	10293.25
18	Diana Mora	2	9217.86
28	Valeria Bravo	2	6881.38
35	Oscar Fernandez	3	6826.81
46	Pedro Fernandez	1	5838.28
54	Veronica Andrade	2	5810.47
48	Ana Morales	1	5685.94
40	Jorge Fernandez	1	5041.94
50	Laura Rios	2	4886.07
52	Natalia Andrade	3	4592.29
33	Alejandro Vega	1	4446.94
19	Valeria Zambrano	1	4264.08
53	Tomas Rodriguez	2	4093.22

Result 2 × Read Only

```sql
-- Consulta 3: Empleados que han realizado al menos una venta
SELECT DISTINCT e.empleado_id, e.nombre, e.cargo
FROM empleados e
JOIN ventas v ON e.empleado_id = v.empleado_id;

-- Consulta 4: Productos con stock bajo (menos de 20 unidades)
SELECT producto_id, nombre, stock
FROM productos
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empleado_id	nombre	cargo
4	Ximena Reyes	Gerente de Ventas
5	Miguel Cevallos	Contador
6	Laura Salgado	Soporte Tecnico
8	Mariana Cevallos	Contador
9	Patricia Espinoza	Jefe de Bodega
10	Cecilia Cevallos	Bodeguero
12	Karina Solis	Jefe de Bodega
14	Pedro Cevallos	Soporte Tecnico
16	Juan Campos	Auxiliar de Inventario
18	Jose Flores	Vendedor
19	Melissa Torres	Cajero
20	Jose Rodriguez	Cajero
21	Melissa Mora	Administrador
22	Camila Cabrera	Jefe de Bodega
23	Esteban Zambrano	Jefe de Bodega
24	Valentina Cabrera	Bodeguero

Result 3 × Read Only

```sql
-- Consulta 4: Productos con stock bajo (menos de 20 unidades)
SELECT producto_id, nombre, stock
FROM productos
WHERE stock < 20
ORDER BY stock;

-- Consulta 5: Top 5 productos mas vendidos (por unidades)
SELECT p.producto_id, p.nombre, SUM(dv.cantidad) AS unidades_vendidas
FROM detalle ventas dv
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

producto_id	nombre	stock
39	Procesador Ryzen 7 Genius	1
20	Gabinete Micro-ATX Gigabyte	4
29	Webcam Full HD BeQuiet	6
24	Monitor 27 pulgadas Curvo D-Link	9
64	Parlantes 2.1 AMD	10
28	Parlantes 2.1 Netgear	17
41	Tarjeta Madre ATX ViewSonic	17
34	Adaptador USB-C ADATA	19
63	Audifonos Gamer 7.1 Apple	19
NULL	NULL	NULL

```sql
-- Consulta 5: Top 5 productos mas vendidos (por unidades)
SELECT p.producto_id, p.nombre, SUM(dv.cantidad) AS unidades_vendidas
FROM detalle_ventas dv
JOIN productos p ON dv.producto_id = p.producto_id
GROUP BY p.producto_id, p.nombre
ORDER BY unidades_vendidas DESC
LIMIT 5;

-- Consulta 6: Clientes que tienen al menos un equipo ensamblado
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

producto_id	nombre	unidades_vendidas
10	SSD 480GB Asus	15
8	Memoria RAM 16GB DDR4 Huawei	15
39	Procesador Ryzen 7 Genius	15
4	Procesador Core i7 Sapphire	13
6	Tarjeta Madre Micro-ATX ViewSonic	13

```sql
-- Consulta 6: Clientes que tienen al menos un equipo ensamblado
SELECT DISTINCT cl.cliente_id, cl.nombre, cl.ciudad
FROM clientes cl
JOIN equipos_ensamblados ee ON cl.cliente_id = ee.cliente_id;

-- Consulta 7: Total comprado a cada proveedor
SELECT pr.proveedor id, pr.nombre, COUNT(c.compra id) AS num compras, SUM(c.total) AS total comprado
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cliente_id	nombre	ciudad
3	Emilio Jimenez	Babahoyo
4	Monica Flores	Babahoyo
5	Monica Herrera	Portoviejo
6	Karina Lopez	Latacunga
9	Sofia Ortiz	Guayaquil
10	Adriana Peña	Machala
14	Fernanda Cevallos	Tulcan
15	Andrea Cevallos	Loja
16	Tomas Solis	Machala
17	Andres Guerrero	Riobamba
22	Julian Flores	Guayaquil
23	Isabella Zambrano	Loja
26	Diego Peña	Manta
27	Raul Espinoza	Babahoyo
28	Valeria Bravo	Riobamba
29	Emilio Reyes	Guayaquil

Result 9 × Read Only

```sql
-- Consulta 7: Total comprado a cada proveedor
SELECT pr.proveedor_id, pr.nombre, COUNT(c.compra_id) AS num_compras, SUM(c.total) AS total_comprado
FROM proveedores pr
JOIN compras c ON pr.proveedor_id = c.proveedor_id
GROUP BY pr.proveedor_id, pr.nombre
ORDER BY total_comprado DESC;

-- ==================================================================
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

proveedor_id	nombre	num_compras	total_comprado
4	Distribuidora Quinde Tech Supply	2	57569.22
20	Distribuidora Castro Global	2	54409.16
2	Distribuidora Cruz Global	3	53866.72
34	Distribuidora Villacis S.A.	2	53068.37
21	Distribuidora Salazar Cia Ltda	2	46213.58
37	Distribuidora Guerrero Import Export	2	45021.82
22	Distribuidora Herrera Import Export	2	35628.99
46	Distribuidora Yepez Import Export	2	35510.54
26	Distribuidora Mendoza Cia Ltda	1	31103.52
38	Distribuidora Peña Tech Supply	2	30475.67
23	Distribuidora Barros Tech Supply	2	29937.34
28	Distribuidora Lopez Tech Supply	1	29822.36
25	Distribuidora Guerrero S.A.	3	29548.07
11	Distribuidora Diaz Cia Ltda	3	26128.12
17	Distribuidora Herrera Import Export	2	24332.45
31	Distribuidora Cabrera Tech Supply	2	21346.68

Result 10 × Read Only

```sql
-- Consulta 9: Productos con precio de venta mayor al promedio de su propia categoria
SELECT p.producto_id, p.nombre, p.categoria_id, p.precio_venta
FROM productos p
WHERE p.precio_venta > (
    SELECT AVG(p2.precio_venta)
    FROM productos p2
    WHERE p2.categoria_id = p.categoria_id
)
ORDER BY p.categoria_id;

-- Consulta 10: Clientes que nunca han realizado una compra (subconsulta NOT EXISTS)
SELECT cl.cliente id, cl.nombre
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

producto_id	nombre	categoria_id	precio_venta
15	Tarjeta de Video RTX 4070 AMD	1	925.15
39	Procesador Ryzen 7 Genius	6	1096.11
36	Microfono Condensador Gigabyte	7	1170.55
30	Impresora Multifuncional Dell	7	1011.84
68	Switch 8 puertos AMD	8	865.53
51	Tarjeta de Video RTX 4070 ViewSonic	8	764.61
53	Fuente de Poder 650W Acer	14	921.58
12	HDD 1TB MSI	15	191.40
44	Memoria RAM 16GB DDR4 BeQuiet	16	656.48
42	Tarjeta Madre Micro-ATX Thermaltake	18	553.12
43	Memoria RAM 8GB DDR4 HP	18	545.90
40	Procesador Core i7 HP	19	1049.75
26	Mouse Optico Gamer AMD	21	802.28
23	Monitor 24 pulgadas ViewSonic	22	1264.93
28	Parlantes 2.1 Netgear	25	463.56
6	Tarjeta Madre Micro-ATX ViewSonic	25	667.89

productos 12

```sql
-- Consulta 10: Clientes que nunca han realizado una compra
SELECT cl.cliente_id, cl.nombre
FROM clientes cl
WHERE NOT EXISTS (
    SELECT 1 FROM ventas v WHERE v.cliente_id = cl.cliente_id
);

-- Consulta 11: Proveedores que no tienen ningun producto regis
```

Result Grid | Filter Rows: | Edit: | Export/Import:

cliente_id	nombre
1	Patricia Rojas
2	Carolina Zambrano
4	Monica Flores
5	Monica Herrera
7	Gabriela Chavez
15	Andrea Cevallos
16	Tomas Solis
20	Alejandro Espinoza
22	Julian Flores
26	Diego Peña
29	Emilio Reyes
30	Natalia Rojas
36	Juan Reyes
37	David Mendoza
38	Julian Flores
39	Valeria Vega

clientes 13

```sql
-- Consulta 11: Proveedores que no tienen ningun producto registrado
SELECT pr.proveedor_id, pr.nombre
FROM proveedores pr
WHERE pr.proveedor_id NOT IN (
    SELECT DISTINCT proveedor_id FROM productos
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

proveedor_id	nombre
3	Distribuidora Pazmiño Import Export
4	Distribuidora Quinde Tech Supply
5	Distribuidora Villacis Import Export
8	Distribuidora Delgado S.A.
12	Distribuidora Gomez Global
17	Distribuidora Herrera Import Export
28	Distribuidora Lopez Tech Supply
29	Distribuidora Villacis Global
33	Distribuidora Villacis Global
36	Distribuidora Cabrera Import Export
43	Distribuidora Barros Cia Ltda
47	Distribuidora Aguilar Import Export
48	Distribuidora Salgado Cia Ltda
NULL	NULL

```sql
-- Consulta 12: Ventas con un total mayor al promedio general de ventas
SELECT venta_id, fecha, cliente_id, total
FROM ventas
WHERE total > (SELECT AVG(total) FROM ventas)
ORDER BY total DESC;

-- Consulta 13: Producto mas caro de cada categoria (subconsulta correla
SELECT p.producto_id, p.nombre, p.categoria_id, p.precio_venta
FROM productos p
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

venta_id	fecha	cliente_id	total
32	2026-03-20	45	8668.47
52	2024-11-10	13	7922.57
35	2025-05-19	11	7887.39
43	2025-08-16	18	7557.69
39	2024-11-13	46	5838.28
2	2024-08-19	48	5685.94
20	2026-08-06	32	5585.85
7	2025-10-13	32	5449.73
14	2026-11-18	28	5057.96
8	2023-06-12	40	5041.94
19	2026-07-22	33	4446.94
48	2024-10-28	19	4264.08
9	2024-11-23	55	4032.82
37	2023-05-28	51	3690.74
54	2025-11-21	17	3688.85
59	2026-01-01	34	3565.80

ventas 15

```
-- Consulta 13: Producto mas caro de cada categoria
SELECT p.producto_id, p.nombre, p.categoria_id, p.precio_venta
FROM productos p
WHERE p.precio_venta = (
    SELECT MAX(p2.precio_venta)
    FROM productos p2
    WHERE p2.categoria_id = p.categoria_id
)
ORDER BY p.categoria_id;

-- Consulta 14: Clientes con mas de 2 ventas registradas
SELECT cl.cliente id, cl.nombre, COUNT(v.venta id) AS num ventas
```

Result Grid | Filter Rows: | Edit: | Export/Import:

producto_id	nombre	categoria_id	precio_venta
15	Tarjeta de Video RTX 4070 AMD	1	925.15
16	Tarjeta de Video RX 6600 Sony	2	931.02
39	Procesador Ryzen 7 Genius	6	1096.11
36	Microfono Condensador Gigabyte	7	1170.55
68	Switch 8 puertos AMD	8	865.53
9	Memoria RAM 32GB DDR5 BeQuiet	9	700.59
17	Fuente de Poder 650W Corsair	12	621.07
48	HDD 1TB Xiaomi	13	234.08
53	Fuente de Poder 650W Acer	14	921.58
12	HDD 1TB MSI	15	191.40
44	Memoria RAM 16GB DDR4 BeQuiet	16	656.48
42	Tarjeta Madre Micro-ATX Thermal...	18	553.12
40	Procesador Core i7 HP	19	1049.75
26	Mouse Optico Gamer AMD	21	802.28
23	Monitor 24 pulgadas ViewSonic	22	1264.93
64	Parlantes 2.1 AMD	23	146.93

productos 16

```
-- Consulta 14: Clientes con mas de 2 ventas registradas
SELECT cl.cliente_id, cl.nombre, COUNT(v.venta_id) AS num_ventas
FROM clientes cl
JOIN ventas v ON cl.cliente_id = v.cliente_id
GROUP BY cl.cliente_id, cl.nombre
HAVING COUNT(v.venta_id) > 2
ORDER BY num_ventas DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cliente_id	nombre	num_ventas
32	Karina Yepez	6
11	Miguel Castro	5
6	Karina Lopez	3
35	Oscar Fernandez	3
45	Tomas Aguilar	3
52	Natalia Andrade	3

```sql
-- Consulta 15: Detalle completo de la venta con el monto mas alto
SELECT dv.*
FROM detalle_ventas dv
WHERE dv.venta_id = (
    SELECT venta_id FROM ventas ORDER BY total DESC LIMIT 1
);

-- Consulta 16: Empleados que han registrado tanto ventas como com
--              (doble subconsulta IN)
```

Result Grid | Filter Rows: | Edit: | Export/Import:

detalle_id	venta_id	producto_id	precio_unitario	cantidad	subtotal
65	32	7	1177.43	2	2354.86
66	32	29	728.41	5	3642.05
67	32	6	667.89	4	2671.56
NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- Consulta 16: Empleados que han registrado tanto ventas como compras
SELECT empleado_id, nombre, cargo
FROM empleados
WHERE empleado_id IN (SELECT empleado_id FROM ventas)
  AND empleado_id IN (SELECT empleado_id FROM compras);

-- Consulta 17: Productos que forman parte de algun equipo ensamblado (s
SELECT producto id, nombre, precio venta
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

empleado_id	nombre	cargo
4	Ximena Reyes	Gerente de Ventas
9	Patricia Espinoza	Jefe de Bodega
10	Cecilia Cevallos	Bodeguero
12	Karina Solis	Jefe de Bodega
14	Pedro Cevallos	Soporte Tecnico
16	Juan Campos	Auxiliar de Inventario
19	Melissa Torres	Cajero
21	Melissa Mora	Administrador
22	Camila Cabrera	Jefe de Bodega
23	Esteban Zambrano	Jefe de Bodega
24	Valentina Cabrera	Bodeguero
25	Pedro Castro	Contador
26	Ricardo Silva	Administrador
27	Alejandro Solis	Cajero
31	Veronica Mendoza	Vendedor
32	Raul Delgado	Administrador

empleados 20 ×

```sql
-- Consulta 17: Productos que forman parte de algun equipo ensamblado
SELECT producto_id, nombre, precio_venta
FROM productos
WHERE producto_id IN (
    SELECT DISTINCT producto_id FROM equipo_componentes
);

-- Consulta 18: Costo total de servicios tecnicos por equipo ensamblado
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

producto_id	nombre	precio_venta
1	Procesador Ryzen 5 Trust	386.96
2	Procesador Core i5 Apple	1080.23
3	Procesador Ryzen 7 NZXT	355.67
4	Procesador Core i7 Sapphire	41.72
5	Tarjeta Madre ATX BeQuiet	353.18
6	Tarjeta Madre Micro-ATX ViewSonic	667.89
7	Memoria RAM 8GB DDR4 HyperX	1177.43
8	Memoria RAM 16GB DDR4 Huawei	248.78
9	Memoria RAM 32GB DDR5 BeQuiet	700.59
10	SSD 480GB Asus	69.13
12	HDD 1TB MSI	191.40
13	HDD 2TB Asus	911.71
14	Tarjeta de Video RTX 4060 Belkin	40.86
15	Tarjeta de Video RTX 4070 AMD	925.15
16	Tarjeta de Video RX 6600 Sony	931.02
17	Fuente de Poder 650W Corsair	621.07

productos 21

```sql
-- Consulta 18: Costo total de servicios tecnicos por equipo ensamblado
SELECT ee.equipo_id, ee.nombre_equipo, costos.total_servicios
FROM equipos_ensamblados ee
JOIN (
    SELECT equipo_id, SUM(costo) AS total_servicios
    FROM equipo_servicios
    GROUP BY equipo_id
) AS costos ON ee.equipo_id = costos.equipo_id
ORDER BY costos.total_servicios DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

equipo_id	nombre_equipo	total_servicios
13	PC Gamer / Oficina #13	202.63
21	PC Gamer / Oficina #21	192.55
32	PC Gamer / Oficina #32	185.45
3	PC Gamer / Oficina #3	164.51
33	PC Gamer / Oficina #33	152.08
43	PC Gamer / Oficina #43	150.04
46	PC Gamer / Oficina #46	146.04
12	PC Gamer / Oficina #12	143.31
52	PC Gamer / Oficina #52	139.51
7	PC Gamer / Oficina #7	136.47
47	PC Gamer / Oficina #47	132.79
50	PC Gamer / Oficina #50	132.71
26	PC Gamer / Oficina #26	129.49
17	PC Gamer / Oficina #17	118.68
15	PC Gamer / Oficina #15	117.50
35	PC Gamer / Oficina #35	109.55

Result 22

```sql
-- Consulta 19: Marca con la mayor cantidad de productos registrados
SELECT m.marca_id, m.nombre, conteo.total_productos
FROM marcas m
JOIN (
    SELECT marca_id, COUNT(*) AS total_productos
    FROM productos
    GROUP BY marca_id
) AS conteo ON m.marca_id = conteo.marca_id
ORDER BY conteo.total_productos DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

marca_id	nombre	total_productos
47	BeQuiet	5

```sql
-- Consulta 20: Clientes cuyo gasto total supera el gasto promedio de todos los clientes
SELECT cl.cliente_id, cl.nombre, SUM(v.total) AS gasto_total
FROM clientes cl
JOIN ventas v ON cl.cliente_id = v.cliente_id
GROUP BY cl.cliente_id, cl.nombre
HAVING SUM(v.total) > (
    SELECT AVG(total_cliente)
    FROM (
        SELECT SUM(total) AS total_cliente
        FROM ventas
        GROUP BY cliente_id
    ) AS sub
)
ORDER BY gasto_total DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cliente_id	nombre	gasto_total
32	Karina Yepez	15281.20
11	Miguel Castro	12067.27
45	Tomas Aguilar	10751.30
13	Ricardo Pazmiño	10293.25
18	Diana Mora	9217.86
28	Valeria Bravo	6881.38
35	Oscar Fernandez	6826.81
46	Pedro Fernandez	5838.28
54	Veronica Andrade	5810.47
48	Ana Morales	5685.94
40	Jorge Fernandez	5041.94
50	Laura Rios	4886.07
52	Natalia Andrade	4592.29

Result 24

- Usuario y roles = 5.

```
84    -- ======================================================================
85    -- 5. VERIFICACION (opcional, util al probar el script)
86    -- ======================================================================
87    SHOW GRANTS FOR 'pedro.quinde'@'%';
88    SHOW GRANTS FOR 'ximena.reyes'@'%';
89    SHOW GRANTS FOR 'miguel.cevallos'@'%';
90    SHOW GRANTS FOR 'gonzalo.ramirez'@'%';
91    SHOW GRANTS FOR 'gabriela.solis'@'%';
92
93
94
```

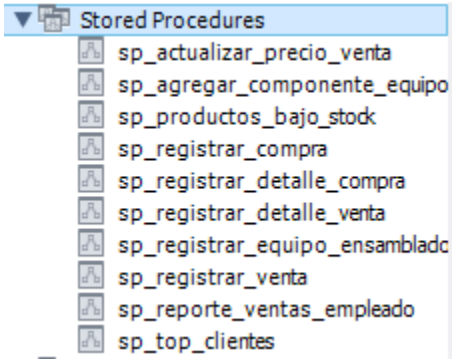
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Grants for gabriela.solis@%
GRANT USAGE ON *.* TO `gabriela.solis`@`%`
GRANT `rol_soporte_tecnico`@`%` TO `gabri...

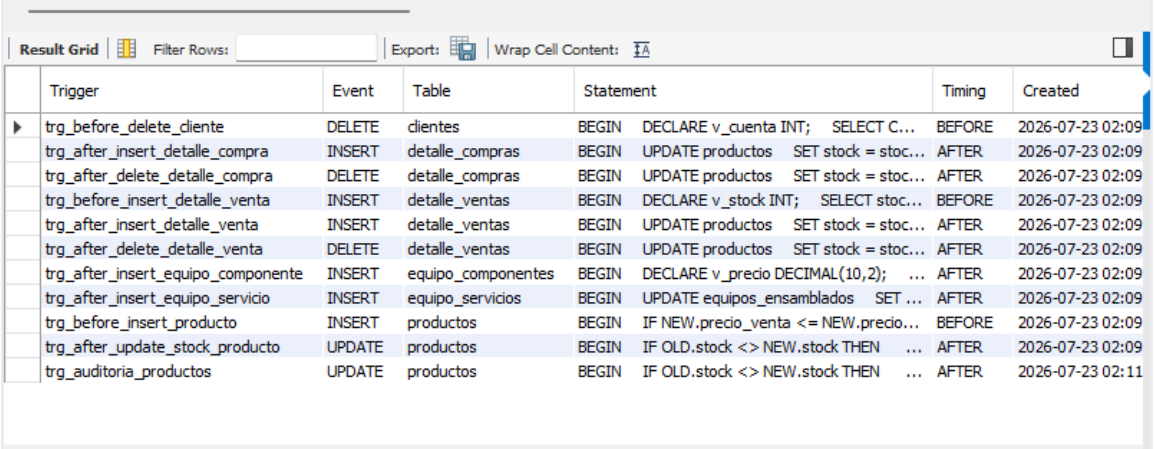
- Vistas = 5.

- Funciones = 10.

- Procedimientos almacenados = 10.



- Triggers = 10.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Trigger	Event	Table	Statement	Timing	Created
trg_before_delete_cliente	DELETE	clientes	BEGIN DECLARE v_cuenta INT; SELECT C...	BEFORE	2026-07-23 02:09
trg_after_insert_detalle_compra	INSERT	detalle_compras	BEGIN UPDATE productos SET stock = stoc...	AFTER	2026-07-23 02:09
trg_after_delete_detalle_compra	DELETE	detalle_compras	BEGIN UPDATE productos SET stock = stoc...	AFTER	2026-07-23 02:09
trg_before_insert_detalle_venta	INSERT	detalle_ventas	BEGIN DECLARE v_stock INT; SELECT stoc...	BEFORE	2026-07-23 02:09
trg_after_insert_detalle_venta	INSERT	detalle_ventas	BEGIN UPDATE productos SET stock = stoc...	AFTER	2026-07-23 02:09
trg_after_delete_detalle_venta	DELETE	detalle_ventas	BEGIN UPDATE productos SET stock = stoc...	AFTER	2026-07-23 02:09
trg_after_insert_equipo_componente	INSERT	equipo_componentes	BEGIN DECLARE v_precio DECIMAL(10,2); ...	AFTER	2026-07-23 02:09
trg_after_insert_equipo_servicio	INSERT	equipo_servicios	BEGIN UPDATE equipos_ensamblados SET ...	AFTER	2026-07-23 02:09
trg_before_insert_producto	INSERT	productos	BEGIN IF NEW.precio_venta <= NEW.precio...	BEFORE	2026-07-23 02:09
trg_after_update_stock_producto	UPDATE	productos	BEGIN IF OLD.stock <> NEW.stock THEN ...	AFTER	2026-07-23 02:09
trg_auditoria_productos	UPDATE	productos	BEGIN IF OLD.stock <> NEW.stock THEN ...	AFTER	2026-07-23 02:11

- Transacciones = 5.

```
-- 1. Ver el encabezado de la última venta registrada
SELECT * FROM ventas
ORDER BY venta_id DESC
LIMIT 1;

-- 2. Ver los detalles de esa venta
SELECT * FROM detalle_ventas
WHERE venta_id = @venta_id;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

venta_id	fecha	cliente_id	empleado_id	total
68	2026-07-23	12	5	761.47
NULL	NULL	NULL	NULL	NULL

```sql
-- 2. Ver los detalles de esa venta
SELECT * FROM detalle_ventas
WHERE venta_id = @venta_id;

-- 3. Ver el stock actualizado de los productos vendido
SELECT producto_id, nombre, stock
```

Result Grid | Filter Rows: | Edit: | Export/Im

detalle_id	venta_id	producto_id	precio_unitario	cantidad	subtotal
128	68	10	69.13	2	138.26
129	68	25	623.21	1	623.21
NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- 3. Ver el stock actualizado de los productos vendidos
SELECT producto_id, nombre, stock
FROM productos
WHERE producto_id IN (10, 25);

-- 4. Ver el registro de auditoría (cambios de stock/precio_venta
SELECT * FROM auditoria_productos
ORDER BY auditoria_id DESC
```

Result Grid | Filter Rows: | Edit: | Export/Import:

producto_id	nombre	stock
10	SSD 480GB Asus	138
25	Teclado Mecanico RGB Crucial	127
NULL	NULL	NULL

```sql
-- 4. Ver el registro de auditoría (cambios de stock/precio_venta)
SELECT * FROM auditoria_productos
ORDER BY auditoria_id DESC
LIMIT 10;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

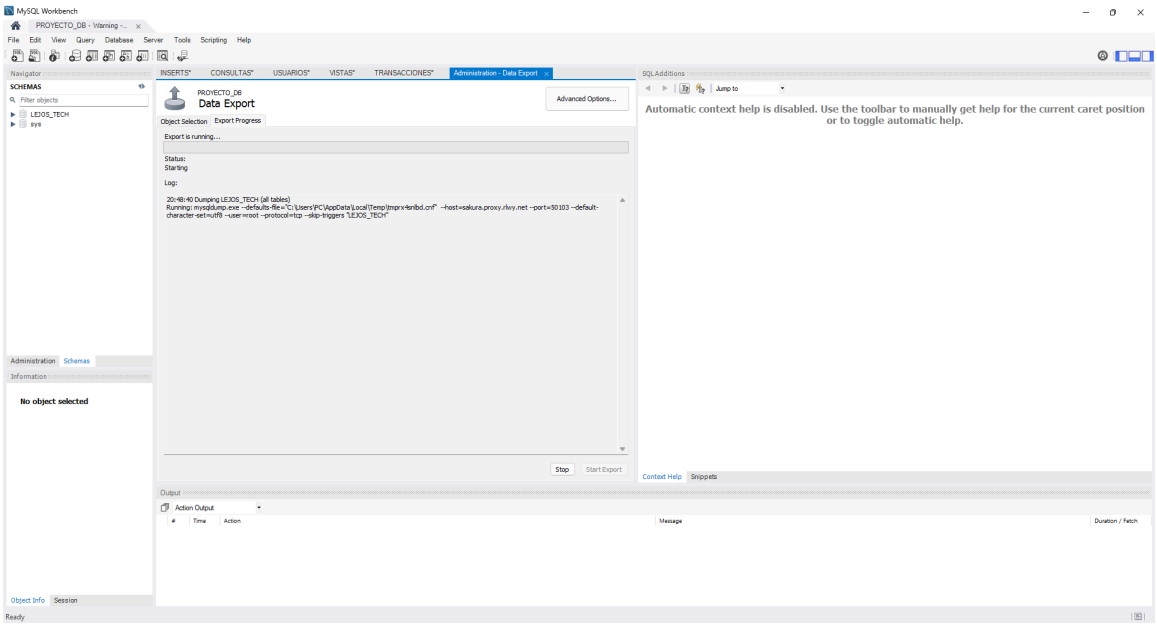
auditoria_id	producto_id	campo_modificado	valor_anterior	valor_nuevo	usuario_bd	fecha_cambio
7	25	stock	117	127	root@%	2026-07-23 02:19:02
6	10	stock	118	138	root@%	2026-07-23 02:19:02
5	25	stock	118	117	root@%	2026-07-23 02:19:00
4	10	stock	120	118	root@%	2026-07-23 02:19:00
3	27	stock	106	105	root@%	2026-07-23 02:14:08
2	18	stock	47	46	root@%	2026-07-23 02:14:08
1	5	stock	29	28	root@%	2026-07-23 02:14:08
NULL	NULL	NULL	NULL	NULL	NULL	NULL

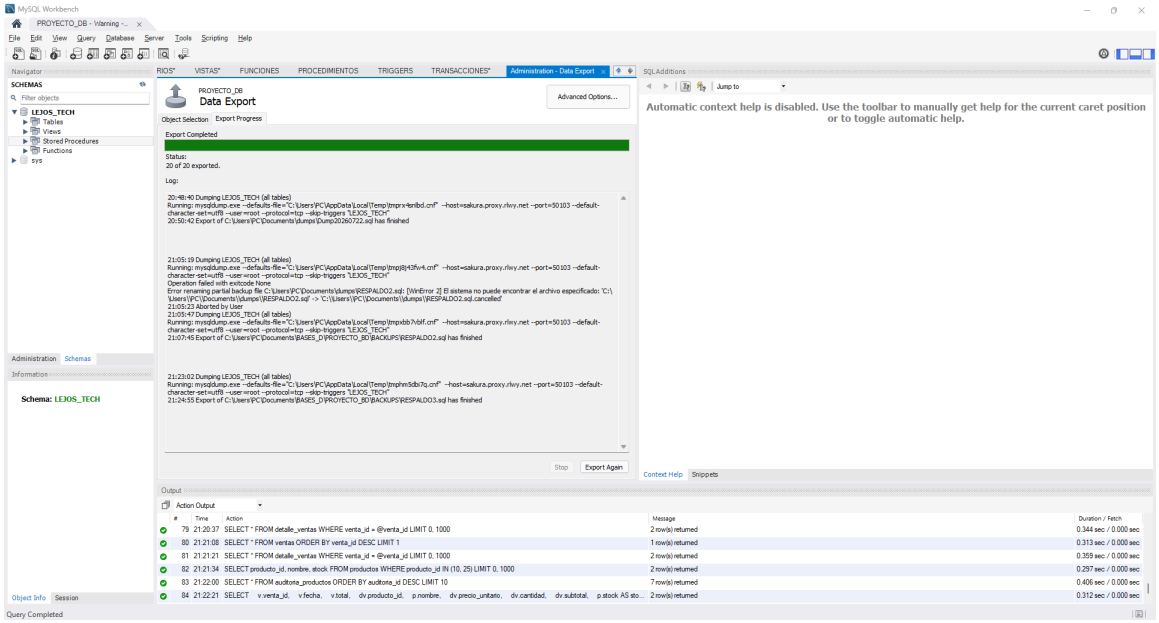
```sql
-- 5. Todo junto en una sola consulta: venta + detalle + producto
SELECT
    v.venta_id,
    v.fecha,
    v.total,
    dv.producto_id,
    p.nombre,
    dv.precio_unitario,
    dv.cantidad,
    dv.subtotal,
    p.stock AS stock_actual
FROM ventas v
JOIN detalle_ventas dv ON v.venta_id = dv.venta_id
JOIN productos p ON dv.producto_id = p.producto_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

venta_id	fecha	total	producto_id	nombre	precio_unitario	cantidad	subtotal	stock_actual
68	2026-07-23	761.47	10	SSD 480GB Asus	69.13	2	138.26	138
68	2026-07-23	761.47	25	Teclado Mecanico RGB Crucial	623.21	1	623.21	127

- Respaldo y recuperación





6. Desarrollo

6.1 Análisis del negocio

6.1.1 Descripción del problema.

LEJOS-TECH, con el propósito de la venta y distribución de componentes para computadoras, requiere gestionar eficientemente su inventario, ventas, compras y logística en un entorno caracterizado por el constante cambio tecnológico y la variación de precios. Entre los problemas identificados se encuentran la falta de control centralizado del inventario, la información dispersa sobre clientes y proveedores, y la imposibilidad de rastrear los cambios de precios. Además, la empresa enfrenta procesos manuales y lentos en pedidos y envíos, carencia de reportes para la toma de decisiones, y un control deficiente de las devoluciones y garantías.

6.1.2 Actores

- Administrador (Control total del sistema).
- Vendedor (Gestión de clientes y ventas).
- Bodeguero (Control de inventario y recepciones).
- Despachador (Gestión de logística y envíos).
- Cliente (Comprador final).

- Proveedor (Suministra hardware).

6.1.3 Reglas del negocio

RN-001: Gestión de Inventario

- Un producto puede tener múltiples ubicaciones de almacenamiento
- El stock no puede ser negativo
- Las existencias se actualizan automáticamente con cada venta/compra

RN-002: Gestión de Clientes

- Cada cliente tiene un único RUC/Cédula
- Un cliente puede tener múltiples direcciones de entrega
- Se registra historial de compras por cliente

RN-003: Gestión de Proveedores

- Cada proveedor tiene un único RUC
- Un proveedor puede suministrar múltiples productos
- Se registra histórico de precios de compra

RN-004: Gestión de Pedidos

- Una orden puede contener múltiples productos
- El estado de la orden puede ser: PENDIENTE, PAGADO, ENVIADO, ENTREGADO, CANCELADO
- Las órdenes de compra requieren aprobación

RN-005: Gestión de Envíos

- Un envío puede contener múltiples órdenes de venta
- Se registra la ruta logística y rastreo
- Los envíos pueden ser: PENDIENTE, EN TRANSITO, ENTREGADO, DEVUELTO

RN-006: Gestión de Precios

- Se registra histórico de cambios de precios
- Existen descuentos por volumen
- Los precios varían por proveedor

RN-007: Auditoría y Seguridad

- Cada usuario tiene un rol específico
- Se registran cambios en inventario

6.1.3 Diccionario de datos

El sistema LEJOS-TECH organiza toda la información de la empresa en 13 tablas principales, diseñadas para que los datos estén seguros, ordenados y sin errores. Para entenderlo de forma sencilla, la información se agrupa en cinco bloques.

- Personal: quién usa el sistema y qué acciones realiza.
- Contactos: clientes y proveedores.
- Inventario: los componentes, sus categorías y los cambios de precio.
- Movimientos: el registro exacto de las compras a proveedores y ventas a clientes.
- Logística: el rastreo de los envíos.

Esta estructura garantiza que no se pierdan datos ni haya información repetida, permitiendo un control total desde que se adquiere un producto hasta que se entrega al cliente final.

6.1.4 Validaciones

1. **Stock No Negativo:** Stock actual ≥ 0
2. **Precios Positivos:** Precio venta > Precio costo
3. **Cantidades Válidas:** Cantidad > 0
4. **Fechas Coherentes:** Fecha entrega ≥ Fecha orden
5. **Montos Positivos:** Total ≥ 0
6. **Descuentos Válidos:** Descuento ≥ 0 y Descuento ≤ Total

7. **Estados Válidos:** Sólo valores permitidos por ENUM
8. **Usuario Activo:** Usuario debe estar ACTIVO para transacciones
9. **Rol Válido:** Usuario debe tener rol asignado

6.2 Diseño de análisis

6.2.1 Modelo Entidad Relación (Mer)

Cuando diseñamos una base de datos, el Modelo Entidad-Relación (MER o ERD) nos sirve como un mapa visual para estructurar la información de manera clara e intuitiva. Propuesta por Peter Chen en 1976, esta técnica nos permite modelar de forma abstracta la arquitectura de los datos, definiendo cómo se agrupan, se vinculan y operan dentro de un sistema.

Para construir este esquema, trabajamos con tres componentes clave que se complementan entre sí:

- **Entidades:** Son los elementos del mundo real (personas, objetos, lugares o conceptos) sobre los que necesitamos guardar registros. Si pensamos en el sistema de una universidad, las entidades serían los alumnos, los docentes, las materias o las aulas.
- **Atributos:** Son los datos concretos que describen y definen a cada entidad. Siguiendo el caso del estudiante, sus atributos serían su número de identificación, nombres, apellidos o la carrera en la que está matriculado.
- **Relaciones:** Son los vínculos lógicos que conectan a las entidades entre sí y explican cómo interactúan dentro del sistema (por ejemplo, cómo un docente *imparte* una asignatura o cómo un alumno se *inscribe* en un curso).

6.2.2 Normalización

Al diseñar una base de datos relacional, la normalización es un proceso clave que aplicamos para estructurar y organizar la información de forma correcta. Su propósito básico es evitar que los datos se repitan innecesariamente (redundancia), prevenir errores

de actualización y garantizar que la base de datos sea eficiente, rápida y fácil de escalar a futuro.

Para lograrlo, aplicamos una serie de reglas paso a paso conocidas como Formas Normales:

- Primera Forma Normal (1NF): Valores atómicos y registros únicos

Es la regla que exige que cada celda de la tabla contenga un solo dato indivisible y que no existan filas repetidas, esta elimina listas o grupos de datos dentro de una misma columna (por ejemplo, guardar varios números de teléfono en una sola casilla) para que cada registro sea único y plano.

- Segunda Forma Normal (2NF): Dependencia total de la clave primaria

Es la regla que aplica sobre la 1NF y asegura que todos los datos de la tabla dependen por completo de la clave principal (Primary Key), esta elimina las dependencias parciales en claves compuestas, sacando a una tabla aparte aquellos atributos que sólo están ligados a una parte de la clave principal y no a toda en su conjunto.

- Tercera Forma Normal (3NF): Cero dependencias entre campos no clave

Es el paso que cumple la 2NF y prohíbe que un campo normal o secundario dependa de otro campo que tampoco sea una clave principal (dependencia transitiva)y esta separa en nuevas tablas aquellos datos que dependen entre sí pero no de la clave primaria (como separar los datos de un departamento que están guardados dentro de la tabla de empleados).

6.2.3 Modelo Lógico

El modelo lógico de datos es una representación detallada de la información que necesita manejar una organización, pero con una característica clave: es completamente independiente del motor de base de datos que vayamos a usar después (como PostgreSQL o MySQL).

En esta fase estructuramos a fondo el sistema. Aquí definimos las entidades, sus atributos, los identificadores únicos (claves), las relaciones con sus restricciones, e incluso jerarquías como supertipos y subtipos. También podemos apoyarnos en glosarios de dominio para mantener los términos estandarizados.

En la práctica, funciona como nuestro plano intermedio. Una vez que tenemos toda esta arquitectura lógica lista y validada, el siguiente paso es transformarla en un modelo físico de datos, que ya es la versión técnica adaptada al gestor de base de datos definitivo.

6.2.4 Modelo Físico

El modelo de datos físico es la implementación técnica y real de una base de datos. A diferencia de los modelos anteriores (que son teóricos y generales), el modelo físico se construye pensando en un motor de base de datos específico (SGBD) y en las necesidades de rendimiento del sistema.

Aquí es donde configuramos físicamente:

- Las tablas y columnas con sus tipos de datos definitivos.
- Las restricciones e índices para optimizar la velocidad de las consultas.
- Las vistas y procedimientos que usarán los programas para interactuar con la información.

En esta etapa ya no solo importa que el diseño sea perfecto en teoría, sino que el sistema sea rápido y funcional. Por ello, a veces es necesario aplicar técnicas como la desnormalización para optimizar los procesos de carga y lectura de datos, garantizando siempre que no se pierda la integridad de la información.

6.2.5 Consultas

Una consulta a una base de datos (lo que comúnmente llamamos query) no es más que una instrucción o petición que le enviamos al gestor para interactuar con la información guardada. Por lo general, escribimos estas peticiones usando SQL (Lenguaje de Consulta Estructurado).

En la práctica, estas consultas se dividen en dos grandes grupos según lo que queramos hacer:

- Consultas de selección (o lectura): Son las que usamos simplemente para buscar, filtrar y extraer datos que cumplan con las condiciones o parámetros que le indiquemos al sistema (básicamente, los clásicos SELECT).
- Consultas de acción (o modificación): Son aquellas que no sólo leen, sino que ejecutan un cambio directo en la estructura o en los registros de la base de datos. En este grupo entran las operaciones de insertar información nueva (INSERT), modificar datos existentes (UPDATE) o eliminar registros (DELETE).

6.2.6 Usuarios y roles

En la administración de bases de datos, los roles son una excelente alternativa a los tradicionales grupos de usuarios, ya que nos permiten gestionar los permisos y privilegios de una forma mucho más flexible y sencilla.

En la práctica, un rol funciona como un contenedor en la base de datos donde agrupamos un conjunto de permisos específicos (por ejemplo, permisos de solo lectura, o de edición sobre ciertas tablas). Una vez que creamos y configuramos este rol con el comando GRANT, podemos asignar fácilmente a usuarios individuales, a grupos enteros, a perfiles generales del sistema o incluso a otros roles para crear jerarquías.

Gracias a esto, en lugar de estar dándole permisos tabla por tabla a cada programador o usuario nuevo, simplemente le asignamos el rol correspondiente y el sistema le otorga de inmediato todo ese bloque de accesos, facilitando enormemente el control de la seguridad.

6.2.7 Vistas

En los gestores de bases de datos, una vista (o view) es básicamente una tabla virtual que creamos a partir del resultado de una consulta SQL. A diferencia de una tabla normal, la vista no almacena la información físicamente en el disco, sino que simplemente organiza

y muestra un conjunto específico de datos provenientes de una o varias tablas reales en el momento en que la consultamos.

Usar vistas en nuestros proyectos nos da tres grandes ventajas prácticas:

1. Nos simplifica el código: Nos permite guardar consultas SQL gigantes (con muchos JOIN o filtros complejos) bajo un solo nombre simple, para no tener que escribirlas una y otra vez.

2. Aumenta la seguridad: Podemos ocultar columnas sensibles o confidenciales y darle acceso a los usuarios o a las aplicaciones únicamente a la información que de verdad necesitan ver.

3. Personaliza la información: Nos ayuda a presentar los datos con el formato o la estructura exacta que requiere la interfaz de nuestro sistema, sin alterar el esquema físico de las tablas originales.

6.2.8 Funciones

En programación y gestión de bases de datos, una función es básicamente un subprograma o rutina que recibe ciertos datos, procesa una operación específica y nos devuelve un resultado concreto. A nivel de sintaxis, siempre la identificamos por su nombre seguido de paréntesis, dentro de los cuales colocamos los datos que va a procesar (a los que llamamos parámetros o argumentos).

El objetivo general de cualquier función es transformar las entradas en un nuevo valor de salida. Un ejemplo clásico en SQL es la función TIMESTAMP(fecha, hora): nosotros le pasamos como argumentos un dato de tipo fecha (DATE) y otro de tipo hora (TIME), y la función procesa esa información para devolvernos un único registro combinado en formato de marca de tiempo (TIMESTAMP).

6.2.9 Procedimientos almacenados

En el desarrollo y administración de bases de datos, los procedimientos almacenados (o stored procedures) son básicamente bloques de código o scripts de SQL que dejamos

guardados directamente en el servidor de base de datos para poder ejecutarlos en cualquier momento.

Su función principal es centralizar la lógica de negocio y automatizar tareas habituales como consultas complejas, inserciones masivas, actualizaciones o validaciones de datos. Usar procedimientos almacenados nos aporta grandes ventajas en un proyecto:

- Evita la repetición de código: En lugar de estar escribiendo las mismas consultas gigantes una y otra vez en las diferentes aplicaciones del sistema, simplemente llamamos al procedimiento ya guardado.
- Mejor rendimiento y escalabilidad: Como el código ya está precompilado en el motor de la base de datos, las operaciones se ejecutan de manera mucho más eficiente y rápida.
- Mayor seguridad y mantenimiento: Podemos restringir que los usuarios o las aplicaciones modifiquen las tablas directamente, dándoles permiso únicamente para ejecutar el procedimiento, lo que protege la integridad y simplifica el mantenimiento del sistema si necesitamos hacer cambios en el futuro.

6.2.10 Triggers

Los triggers o disparadores son herramientas poderosas en bases de datos relacionales que permiten automatizar acciones ante ciertos eventos, sin necesidad de intervención manual. Aunque su concepto puede parecer simple, su correcta implementación puede mejorar la integridad de datos y simplificar la lógica de negocio en tus sistemas.

6.2.11 Transacciones

En los sistemas de gestión de bases de datos (SGBD o DBMS), una transacción es básicamente una secuencia o bloque de operaciones (como inserciones, actualizaciones o borrados) que el sistema trata como una única unidad lógica de trabajo indivisible. Para que una transacción sea válida, o se ejecutan todas sus tareas de forma exitosa y se

guardan los cambios (COMMIT), o no se aplica ninguna y el sistema vuelve al estado anterior (ROLLBACK).

6.2.12 Seguridad y respaldos

En el desarrollo de software y la administración de sistemas, la seguridad de bases de datos engloba todo el conjunto de estrategias, herramientas tecnológicas y controles de acceso que implementamos para proteger los repositorios de información frente a ataques o filtraciones.

7. Presentación

7.1 Informe PDF

El proyecto será entregado en formato PDF, el cual contiene el desarrollo completo del análisis, diseño e implementación de la base de datos de LEJOS-TECH, junto con los diagramas, scripts y capturas de evidencia correspondientes a cada fase del desarrollo.

7.2 Repositorio GitHub

Todo el código desarrollado (script DDL, consultas, vistas, funciones, procedimientos almacenados, triggers y respaldos) se encuentra disponible en el siguiente repositorio de GitHub:

https://github.com/Mateojrq/Proyecto_BD.git

El repositorio se mantiene organizado por carpetas según el tipo de objeto de base de datos, facilitando la revisión y ubicación de cada componente del proyecto.

7.3 Base de datos en la nube

Para efectos de revisión, la base de datos se encuentra alojada en la nube y es accesible mediante MySQL Workbench, con las siguientes credenciales de conexión:

- **Nombre de conexión:** PROYECTO_DB
- **Hostname:** **sakura.proxy.rlwy.net**
- **Puerto:** 50103

- **Usuario:** root
- **Password:** kTMwKFVYtCNnCJmXtZXMeRabhaUJChgs

7.4 Defensa oral

Se realizará una defensa oral del proyecto, en la cual se presentará una breve diapositiva explicando los aspectos más relevantes del desarrollo (modelo de datos, decisiones de diseño y objetos implementados), seguida de la ejecución en vivo del código para demostrar el funcionamiento real de las consultas, triggers, procedimientos y demás componentes de la base de datos.

8. Recursos necesarios

Para el desarrollo del presente proyecto se utilizaron las siguientes herramientas:

- MySQL Workbench: entorno gráfico utilizado para el diseño del modelo físico, la ejecución de los scripts SQL y la administración general de la base de datos (creación de tablas, vistas, funciones, procedimientos y triggers).
- Railway: plataforma web utilizada para el despliegue y alojamiento del servidor de base de datos MySQL en la nube, permitiendo el acceso remoto a la base de datos desde MySQL Workbench.
- Draw.io: herramienta utilizada para el diseño del Modelo Entidad-Relación (MER) a nivel conceptual.
- Microsoft Excel: utilizado para la organización y generación de los datos de prueba (mínimo 50 registros por tabla) previo a su carga en la base de datos.
- GitHub: plataforma utilizada para el control de versiones y almacenamiento del repositorio del proyecto, donde se encuentra disponible el código fuente completo.

9. Conclusiones

El desarrollo de este proyecto permitió aplicar de forma práctica todo el proceso de diseño de una base de datos relacional, desde el análisis inicial del negocio hasta la implementación física en MySQL. Al trabajar con un caso real cómo LEJOS-TECH, se

pudo entender mejor por qué es tan importante hacer un buen levantamiento de reglas de negocio antes de empezar a crear tablas, ya que varias decisiones del modelo (como el manejo de estados de pedidos o el historial de precios) surgieron directamente de las necesidades identificadas en esa etapa.

La normalización fue una de las partes que más ayudó a comprender la lógica detrás del diseño relacional, ya que al pasar la información desde una tabla sin normalizar hasta la 3FN se evidenció claramente cómo se eliminan las redundancias y se evitan las anomalías de actualización, algo que en la teoría se ve simple pero que al aplicarlo con datos reales cuesta más trabajo de lo que parece.

En cuanto a la implementación, el uso de vistas, funciones, procedimientos almacenados y triggers permitió automatizar varias reglas de negocio directamente en la base de datos, como el control de stock o la actualización automática de totales, reduciendo así la dependencia de que esa lógica se controle manualmente o desde una aplicación externa. Esto demuestra que un buen diseño de base de datos no solo organiza la información, sino que también puede encargarse de mantener su integridad de forma automática.

Finalmente, trabajar con una base de datos alojada en la nube (Railway) y administrada desde MySQL Workbench permite simular un entorno más cercano al que se usaría en un ambiente laboral real, donde el acceso remoto, el control de usuarios y roles, y las políticas de respaldo son parte fundamental de la administración de cualquier sistema de información.

10. Recomendaciones

Antes de empezar a crear las tablas, conviene sentarse a conversar bien sobre las reglas de negocio con calma, porque varias veces durante el proyecto tocó regresar a modificar el modelo porque algo no se había pensado bien al inicio (por ejemplo, cómo manejar los distintos estados de un pedido o qué pasa cuando un proveedor deja de suministrar un

producto). Tener esto más claro desde el principio ahorra bastante trabajo de corrección después.

Es recomendable ir probando cada objeto de la base de datos (vista, función, procedimiento, trigger) apenas se crea, en lugar de dejar todo para probarlo junto al final. Cuando se prueba todo de golpe es más difícil identificar en qué parte específica está el error, sobre todo si un trigger depende de un procedimiento o una función que también se acaba de crear.

Para la carga de los 50 registros mínimos por tabla, es mejor generar primero las tablas que no dependen de otras (como categorías o roles) y luego ir llenando las que sí tienen llaves foráneas, respetando ese orden. Intentar cargar todo al mismo tiempo sin seguir esa jerarquía genera errores de referencia que consumen tiempo innecesario.

También se recomienda no dejar la documentación del proyecto para el final. Ir anotando qué hace cada consulta, vista o procedimiento mientras se va programando ayuda mucho al momento de armar el informe, porque de lo contrario hay que volver a leer el código para recordar la lógica de cada uno.

Por último, se recomienda revisar bien las relaciones CASCADE y RESTRICT antes de cargar los datos de prueba, ya que un mal uso de estas reglas puede borrar o bloquear registros relacionados sin que uno se dé cuenta, especialmente cuando se están haciendo pruebas de eliminación en tablas con muchas dependencias.

11. Bibliografía

Arzola, A. (2025, 09 29). Triggers en Bases de Datos: Automatizando la Gestión de Datos. Triggers en Bases de Datos: Automatizando la Gestión de Datos. https://medium.com/@Alejandro.D.A.S/triggers-en-bases-de-datos-automatizando-la-gesti%C3%B3n-de-datos-060eb4dd1791.

Ballejos, L. (2024, February 1). ¿Qué es una consulta de base de datos? NinjaOne. Retrieved July 11, 2026, from https://www.ninjaone.com/es/it-hub/it-service-management/consulta-de-base-de-datos-que-es/.

DREMIO. (n.d.). Transacción de base de datos. Transacción de base de datos. https://www.dremio.com/wiki/database-transaction/

DREMIO. (n.d.). Vistas de la base de datos. Vistas de la base de datos. https://www.dremio.com/wiki/database-views/

IBM. (n.d.). ¿Qué es la seguridad de la base de datos? ¿Qué es la seguridad de la base de datos? https://www.ibm.com/es-es/think/topics/database-security

IBM. (2021, 03 08). Modelos lógicos de datos. Modelos lógicos de datos. https://www.ibm.com/docs/es/ida/9.1.2?topic=modeling-logical-data-models

IBM. (2024, 01 05). Funciones. Funciones. https://www.ibm.com/docs/es/db2/11.1.0?topic=elements-functions

IBM. (2024, 01 25). Roles. Roles. https://www.ibm.com/docs/es/db2/11.1.0?topic=security-roles

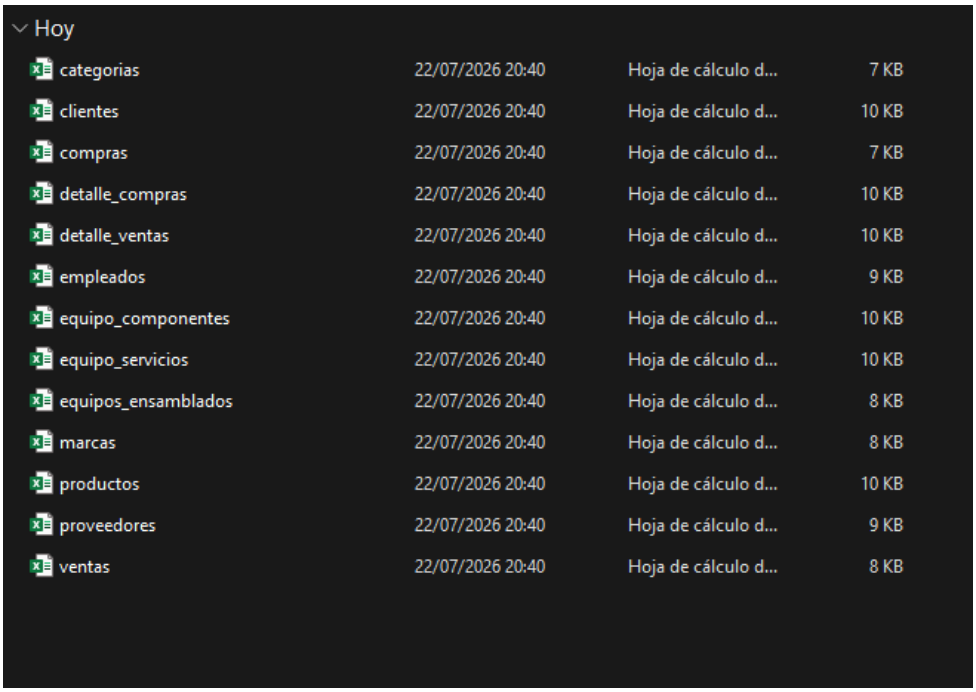
Ouko, A. (2025, February 14). Procedimiento almacenado SQL: Automatizar y optimizar consultas. DataCamp. Retrieved July 11, 2026, from https://www.datacamp.com/es/tutorial/sql-stored-procedure

Quest. (n.d.). Modelado de datos físico. Modelado de datos físico. https://www.quest.com/es-mx/solutions/data-modeling/physical.aspx

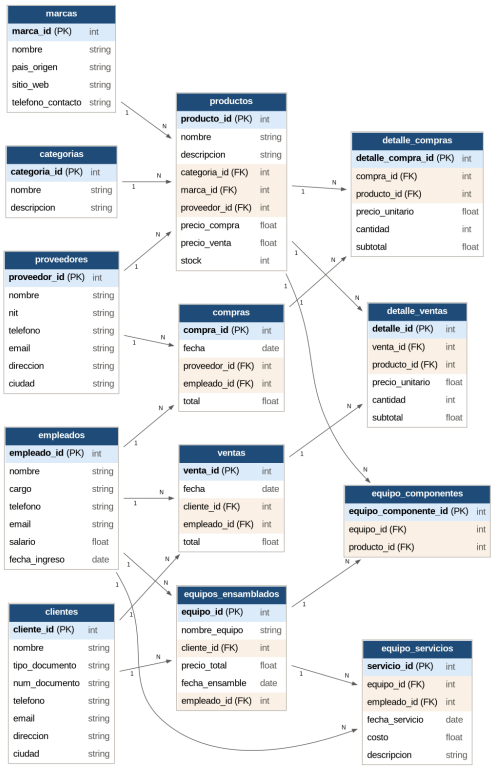
Universidad Europea. (2024, September 9). Normalización de base de datos | Blog UE. Normalización de base de datos | Blog UE. Retrieved July 11, 2026, from https://universidadeuropea.com/blog/normalizacion-bases-datos/

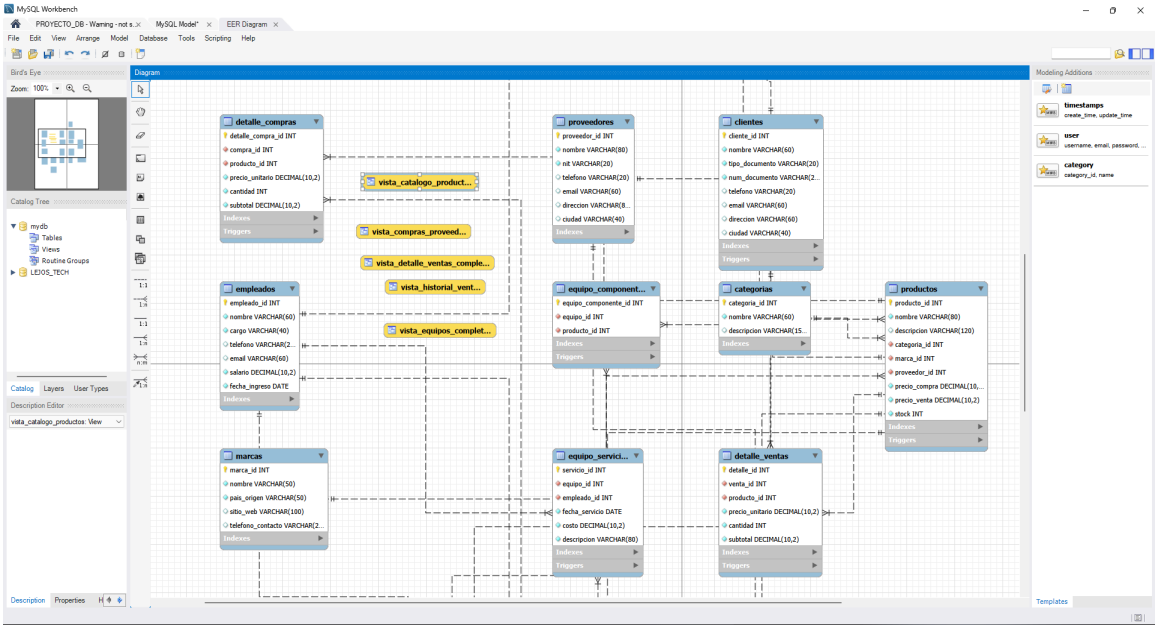
Universidad Europea. (2025, June 13). Modelo Entidad Relación (MER o ERD) qué es y proceso de creación | Blog UE. Modelo Entidad Relación (MER o ERD) qué es y proceso de creación | Blog UE. Retrieved July 11, 2026, from https://universidadeuropea.com/blog/modelo-entidad-relacion/

12. Anexos



Hoy			
categorias	22/07/2026 20:40	Hoja de cálculo d...	7 KB
clientes	22/07/2026 20:40	Hoja de cálculo d...	10 KB
compras	22/07/2026 20:40	Hoja de cálculo d...	7 KB
detalle_compras	22/07/2026 20:40	Hoja de cálculo d...	10 KB
detalle_ventas	22/07/2026 20:40	Hoja de cálculo d...	10 KB
empleados	22/07/2026 20:40	Hoja de cálculo d...	9 KB
equipo_componentes	22/07/2026 20:40	Hoja de cálculo d...	10 KB
equipo_servicios	22/07/2026 20:40	Hoja de cálculo d...	10 KB
equipos_ensamblados	22/07/2026 20:40	Hoja de cálculo d...	8 KB
marcas	22/07/2026 20:40	Hoja de cálculo d...	8 KB
productos	22/07/2026 20:40	Hoja de cálculo d...	10 KB
proveedores	22/07/2026 20:40	Hoja de cálculo d...	9 KB
ventas	22/07/2026 20:40	Hoja de cálculo d...	8 KB





Github: https://github.com/LeitoM10/Moreira_Rodriguez_M_PROYECTO_BD